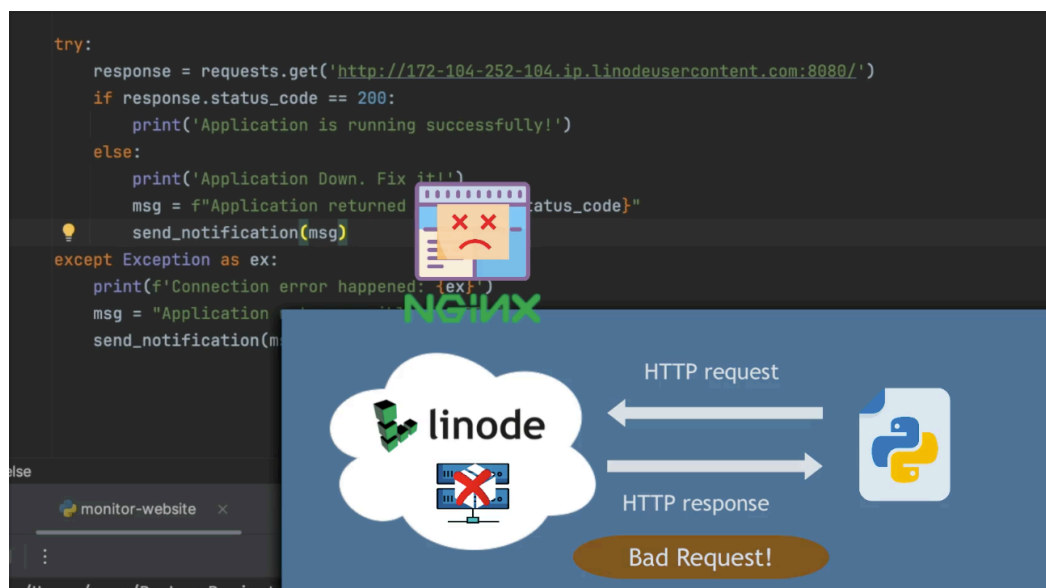
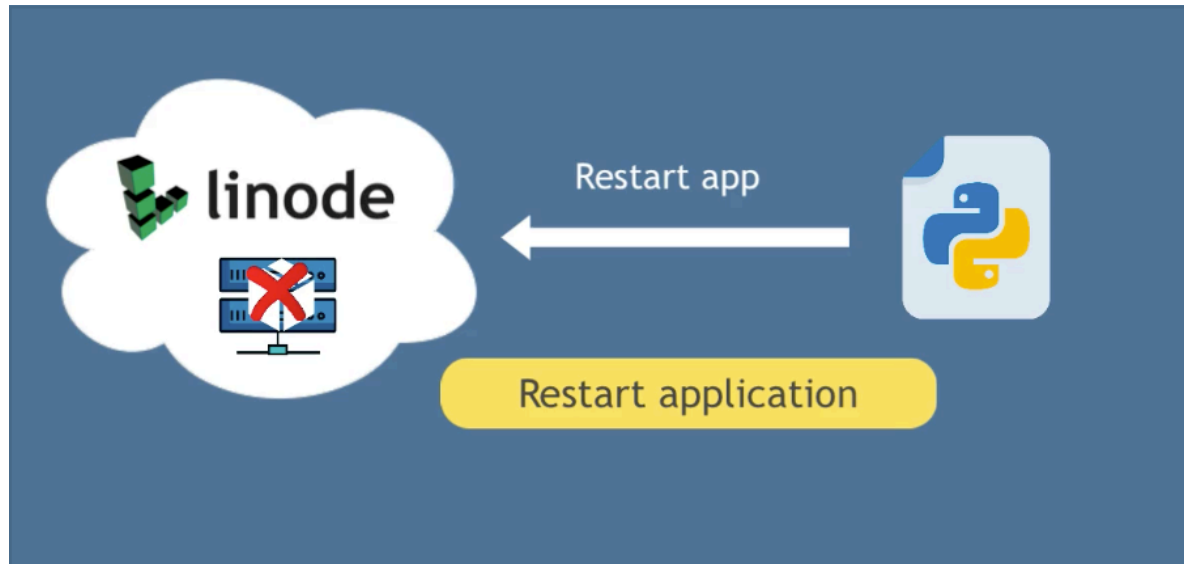


Module 14 - chapter 15

Website Monitoring 3: Restart Application and Reboot Server

Restart the application

So after notifying the app is down we want to restart the app automatically.



So if the app falls into lines 22 - 25 then just restating the app may help.

So we will write logic to restart the nginx docker container.

And python will connect to Linode server via ssh



And we will use a library called Paramiko

Paramiko

- Python implementation of SSHv2
- Library for making SSH connections (client or server)

The image shows the Paramiko logo, which is a lightbulb with a yellow glow. Below the logo, the word 'Paramiko' is written in a large, bold, white font. Underneath, there are two bullet points in white text: '- Python implementation of SSHv2' and '- Library for making SSH connections (client or server)'.

pypl.org/project/paramiko/

BC Player - Home | Trello | Wise - Login | Google NotebookL... | Overleaf - CV | CodeSignal Learn | Cloud Linode | Linux | Bash | coding | Java | DevOps | python | Azure | NatCen

Type '/' to search projects

Help Docs Sponsors Log in Register

paramiko 5.0.0 [Latest release]

Released: May 9, 2026

SSH2 protocol library

Navigation

- Project description
- Release history
- Download files

Verified details

These details have been verified by PyPI

Maintainers

- bitprophet

Unverified details

These details have not been verified by PyPI

Project links

- Changelog

Project description

paramiko 5.0.0 | python 3.9 | 3.10 | 3.11 | 3.12 | 3.13 | license LGPL-2.1 | build failing | coverage 85%

Welcome to Paramiko!

Paramiko is a pure-Python [1] implementation of the SSHv2 protocol [2], providing both client and server functionality. It provides the foundation for the high-level SSH library [Fabric](#), which is what we recommend you use for common client use-cases such as running remote shell commands or transferring files.

Direct use of Paramiko itself is only intended for users who need advanced/low-level primitives or want to run an in-Python sshd.

For installation information, changelogs, FAQs and similar, please visit [our main project website](#); for API details, see [the versioned docs](#). Additionally, the project maintainer keeps a [roadmap](#) on his personal site.

[1] Paramiko relies on [cryptography](#) for crypto functionality, which makes use of C and Rust extensions but has many precompiled options available. See [our installation page](#) for details.

[2] OpenSSH's RFC specification page is a fantastic resource and collection of links that we won't bother replicating here: <https://www.openssh.com/specs.html> OpenSSH itself also happens to be our primary reference implementation: when in doubt, we consult how they do things, unless there are good reasons not to. There are always some gaps, but we do our best to reconcile them when possible.

-> pip install paramiko

```

> pip install paramiko
Collecting paramiko
  Downloading paramiko-5.0.0-py3-none-any.whl.metadata (3.7 kB)
Collecting bcrypt>=3.2 (from paramiko)
  Downloading bcrypt-5.0.0-cp39-abi3-macosx_10_12_universal2.whl.metadata (10 kB)
Collecting cryptography>=3.3 (from paramiko)
  Downloading cryptography-49.0.0.tar.gz (854 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 854.3/854.3 kB 21.8 MB/s 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Installing backend dependencies ... done

```

So now we import the paramiko library into our code and we create an ssh client

```

26         send_notification(msg)
27
28         # restart the application
29 ✨ ssh = paramiko.SSHClient() You, 6 minutes ago
30
31
32     except Exception as ex:

```

And now we make the connection of the client like we do here:

```

~ ➤ ssh root@172.236.15.151

```

So .connect() has many parameters and we need to provide the hostname which is the first parameter

```

def connect(self,
            hostname: str,
            port: int = 22,
            username: str | None = None,
            password: str | None = None,
            pkey: PKey | None = None,
            key_filename: str | None = None,

```

-> .connect("172.236.15.151")

Next we pass the port, by default is 22 and that is the port we need so there is no need to provide it.

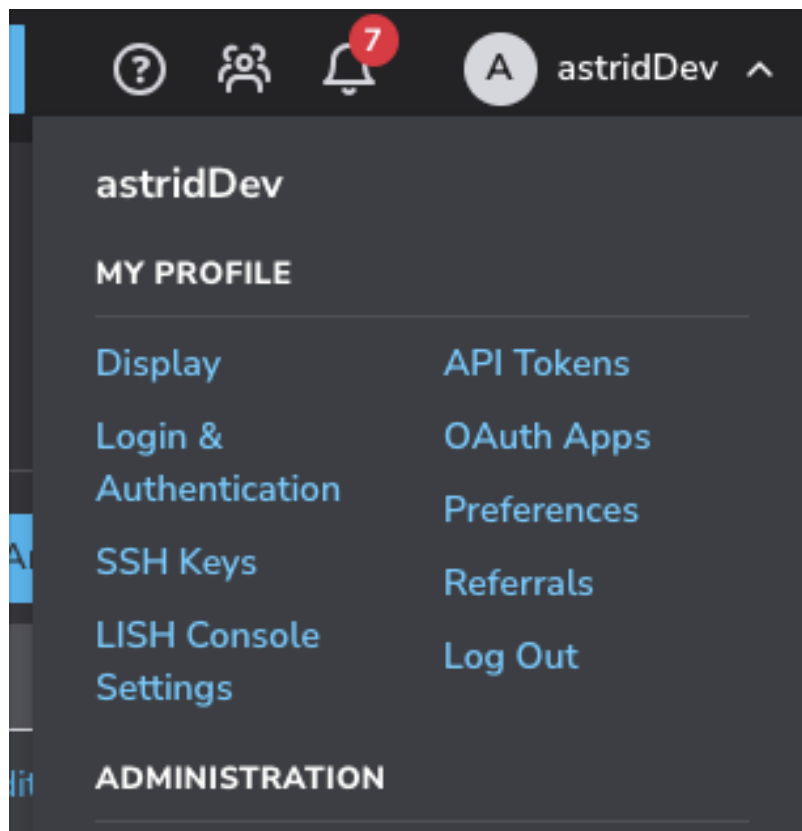
Next we pass the username, this is the third parameter and we are passing it as the second argument so I need to let python know the name of the parameter as I am now not longer passing the arguments in the same order as the parameters.

```
-> .connect("172.236.15.151", username="root")
```

Now I can pass a password but we created a ssh key in Linode that uses a public key that we have in our .ssh folder

```
> cd ~/.ssh
> ls
config          id_ed25519
docker-server.pem id_ed25519.pub
> cat id_ed25519.pub
```

Which I can see here in Linode



So when we ssh into our Linode server we use our private ssh which is then

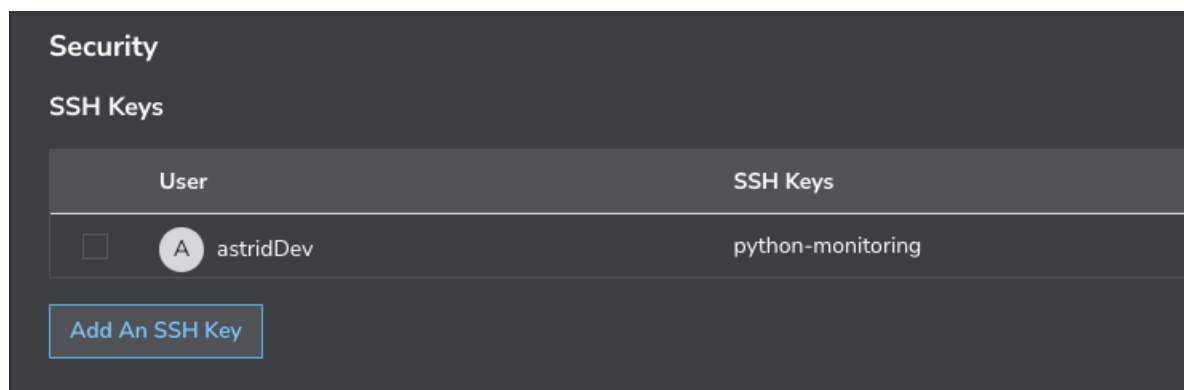
match with the public ssh that is stored in Linode

-> `ssh -i ~/.ssh/id_ed25519 root@172.236.15.151` is the same as -> `ssh root@172.236.15.151`

So in the terminal we were passing the private key because it is the default file in the default location but we didn't see it. But for python we need to tell it where the private key is, we give python the whole path to go and find the private key:

```
28 # restart the application
29 ssh = paramiko.SSHClient()
30 ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
31
```

Actually when I ssh from my laptop into the Linode server it asks for the password every single time, I realised that it is because when I created the server I created the ssh key but I didn't select it



So the ssh key was not linked to this server (see notes in the questions section), it is there in my Linode user profile but is not linked to any server.

So next time I need to check the box.

For now the fix:

-> ssh into the Linode server

-> check `~/.ssh` maps to 700

`drwx----- 2 root root 4096 Jul 11 08:46 .ssh`

-> check `~/.ssh/authorized_keys` maps to 600

`-rw----- 1 root root 117 Jul 11 15:00 authorized_keys`

-> `echo "ssh-ed25519 AAAA...paste-your-full-public-key-here... python-monitoring" >> ~/.ssh/authorized_keys`

-> check `~/.ssh/authorized_keys`

And all is good now

-> exit

Test again and it works, this time no password is asked

```
> ssh root@172.236.15.151
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 11 03:15:53 PM UTC 2026

System load:          0.08
Usage of /:           6.7% of 48.63GB
Memory usage:         13%
Swap usage:           0%
Processes:            109
Users logged in:      0
IPv4 address for eth0: 172.236.15.151
IPv6 address for eth0: 2600:3c13::2000:8dff:fee0:1ae3

Expanded Security Maintenance for Applications is not enabled.

86 updates can be applied immediately.
76 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Jul 11 14:39:16 2026 from 140.228.47.252
root@localhost:~#
```

Now that python can connect without the password issue we can execute linux commands:

For example

-> docker ps

And we will get in return (see questions section for more details)

-> stdin -> standard input

-> stdout -> standard output

-> stderr. -> standard error

```
28 # restart the application
29 ssh = paramiko.SSHClient()
30 ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
31 stdin, stdout, stderr = ssh.exec_command("docker ps")
32 print(stdin)
33 print(stdout) You, Moments ago • Uncommitted changes
```

Another important thing is that when we ssh for the first time we get a question "are you sure you want to continue connecting (yes/no[fingerprint])?" Asking if we want to add a missing key basically:

```
> ssh root@172.236.15.151
The authenticity of host '172.236.15.151 (172.236.15.151)' can't be established.
ED25519 key fingerprint is SHA256:an03sIvdHBn/E1Juki4UjrIErj4evNj4V43SZj9hLU4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.236.15.151' (ED25519) to the list of known hosts.
root@172.236.15.151's password:
```

The above is in interactive mode "-l"

But python can't do that it needs to be done automatically and for that we use

-> `.set_missing_host_key_policy(paramiko.AutoAddPolicy())`

To add the host key to the server.

Now we have this code:

```
17
18
19     try:
20         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
21         if response.status_code == 200:
22             print("Application is running successfully!")
23         else:
24             print("Application Down. Fix it!")
25             # send notification email to support
26             msg = f"Application returned {response.status_code}"
27             send_notification(msg)
28
29             # restart the application
30             ssh = paramiko.SSHClient()
31             ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
32             ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astri
33             stdin, stdout, stderr = ssh.exec_command("docker ps")
34             print(stdin)
35             print(stdout)
```

And we can test the script by making sure the nginx container is running and that it falls in the 'else' statement

```
18     try:
19         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
20         if False: Caballero-Pardo, Today • Monitor app access is succesful and add README
21             print("Application is running successfully!")
22         else:
23             print("Application Down. Fix it!")
24             # send notification email to support
25             msg = f"Application returned {response.status_code}"
26             send_notification(msg)
27
28             # restart the application
29             ssh = paramiko.SSHClient()
30             ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
31             ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/ast
32             stdin, stdout, stderr = ssh.exec_command("docker ps")
33             print(stdin)
34             print(stdout)
```

```
monitor-website.py
31 ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/a
32 stdin, stdout, stderr = ssh.exec_command("docker ps")
33 print(stdin)
34 print(stdout)
35 You, Moments ago • Uncommitted changes
36 except Exception as ex:
37     print(f"Connection error happened: {ex}")
38     # send notification email to support
39     msg = "Application not accessible at all "
```

```
Run monitor-website
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automati
Application Down. Fix it!
<paramiko.ChannelFile from <paramiko.Channel 0 (open) window=2097152 -> <paramiko.Transport at 0xfcc4830
<paramiko.ChannelFile from <paramiko.Channel 0 (open) window=2097152 -> <paramiko.Transport at 0xfcc4830
```

We only get stdout so let's print it but we also need to decode stdout as currently is in bytes format and we want to be able to read it so we will use `.readlines()`

```
28 # restart the application
29 ssh = paramiko.SSHClient()
30 ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
31 ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
32 stdin, stdout, stderr = ssh.exec_command("docker ps")
33 print(stdout.readlines()) You, 5 minutes ago • Uncommitted changes
34
35 except Exception as ex:
36     print(f"Connection error happened: {ex}")
37     # send notification email to support
```

```
Run monitor-website
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
Application Down. Fix it!
['CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS
Process finished with exit code 0
```

And the other part of the output from stdout

```
33 print(stdout.readlines()) You, 5 minutes ago • Uncommitted changes
34
35 except Exception as ex:
36     print(f"Connection error happened: {ex}")
37     # send notification email to support
```

```
Run monitor-website
nitor-website.py
NAMES\n', '52527a5428e8  nginx  "/docker-entrypoint..."  6 hours ago  Up 7 minutes  0.0.0.0:8080->{
```

It works! And when we are done we close the connection see line 34 📌
-> `.close()`

```

27
28     # restart the application
29     ssh = paramiko.SSHClient()
30     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
31     ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
32     stdin, stdout, stderr = ssh.exec_command("docker ps")
33     print(stdout.readlines())
34     ssh.close()
35
36 except Exception as ex:
37     print(f"Connection error happended: {ex}")

```

Now what we want is to restart the application so we want to execute this command:

-> docker start 52527a5428e8

And for now we will hardcoded it:

```

18 try:
19     response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
20     if response.status_code == 200:
21         print("Application is running successfully!")
22     else:
23         print("Application Down. Fix it!")
24         # send notification email to support
25         msg = f"Application returned {response.status_code}"
26         send_notification(msg)
27
28         # restart the application
29         ssh = paramiko.SSHClient()
30         ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
31         ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
32         stdin, stdout, stderr = ssh.exec_command("docker start 52527a5428e8")
33         print(stdout.readlines())
34         ssh.close()
35

```

Reboot the server

We also want to restart the server if we can't connect at all:



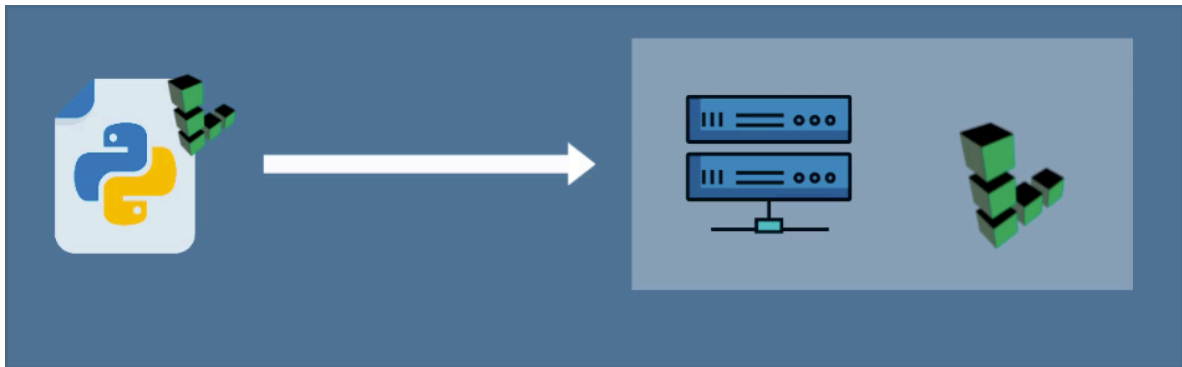
So we will need to restart the server and then we will restart the application:

```

37     except Exception as ex:
38         print(f"Connection error happened: {ex}")
39         # send notification email to support
40         msg = "Application not accessible at all."
41         send_notification(msg)
42
43         # restart linode server
44
45         # restart the application  You, Moments ago • Uncommi
46

```

So to restart the server, first we need python Linode client to be able to connect to the Linode account and then reboot the Linode server



And for that we will use python's Linode library Called `linode_api4`

The screenshot shows the PyPI project page for **linode-api4 5.45.0**. The page header includes the project name and version, along with a search bar and navigation links. The main content area is divided into sections: **Navigation** (Project description, Release history, Download files), **Project description** (linode_api4, The official python library for the Linode API v4 in python, tests repo or workflow not found, pip package 5.45.0, docs passing), **Installation** (pip install linode_api4), **Building from Source** (To build and install this package: Clone this repository, python3 -m pip install .), **Verified details** (These details have been verified by PyPI), **Project links** (Homepage, Repository), and **GitHub Statistics** (Repository, Stars: 142).

And we install it

```
> pip install linode-api4
Collecting linode-api4
  Downloading linode_api4-5.45.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: requests in ./venv/lib/python3.14/site-packages
Collecting polling (from linode-api4)
  Downloading polling-0.3.2.tar.gz (5.2 kB)
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Collecting deprecated (from linode-api4)
  Downloading deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting wrapt<3, >=1.10 (from deprecated->linode-api4)
```

So now we import the library and create a Linode client which will make it possible for python to connect to Linode.

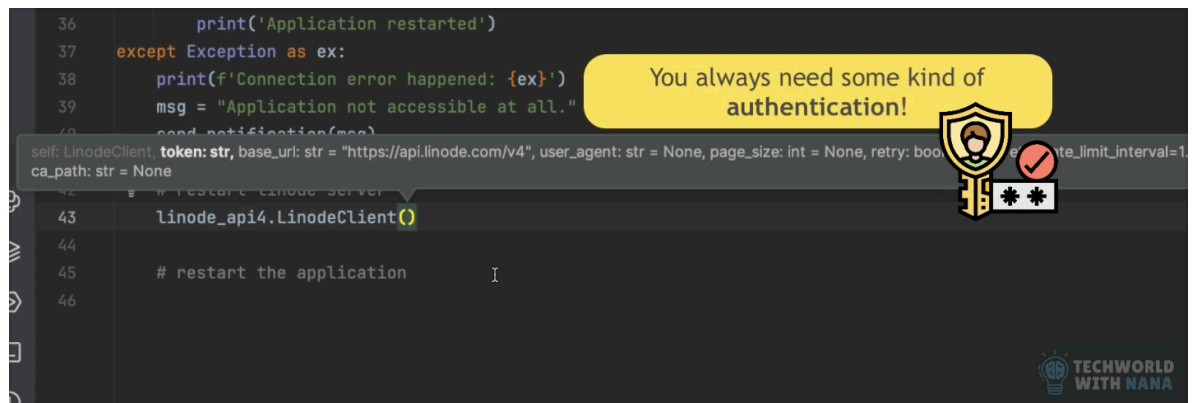
-> .LinodeClient()

```

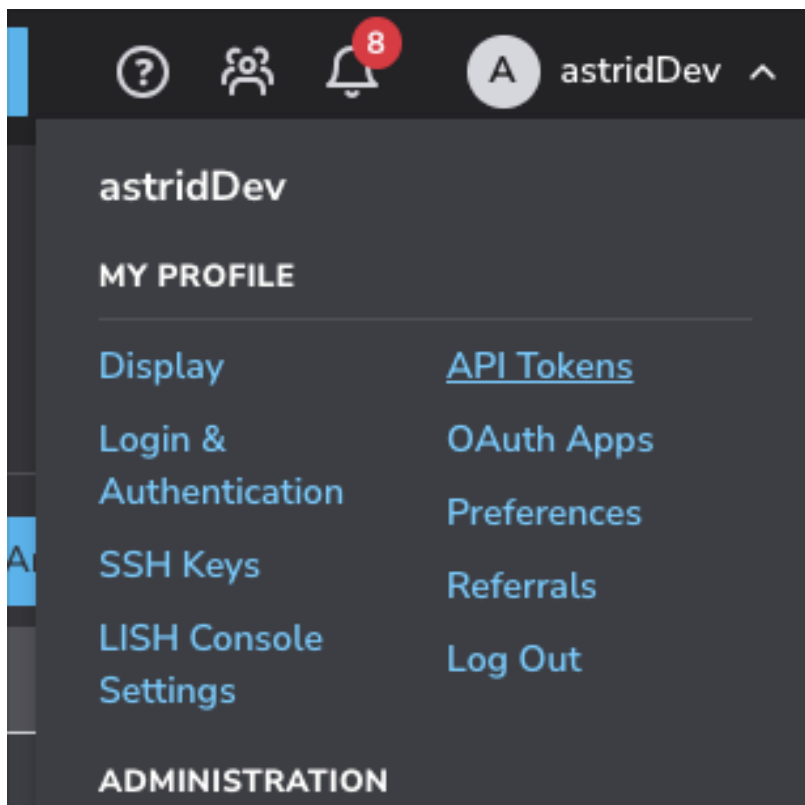
38 except Exception as ex:
39     print(f"Connection error happened: {ex}")
40     # send notification email to support
41     msg = "Application not accessible at all."
42     send_notification(msg)
43
44     # restart linode server
45     💡 linode_api4.LinodeClient()
46     | You, A minute ago • Uncommitted changes

```

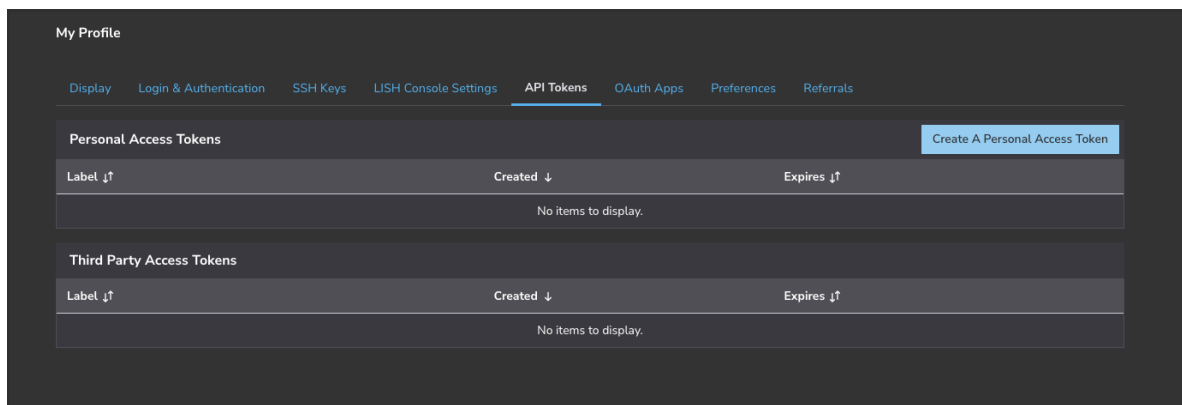
And we need to pass a Linode token as an argument as we always need some kind of authentication of the external party in this case python to be able to access Linode account.



In Linode console we can find API tokens



And in there we can create the API tokens that then I can give to the third party like python to use to authenticate to Linode account.



We give it a name 'python' and give read/write permissions so we can reboot the server (we need write permission)

Add Personal Access Token

Label

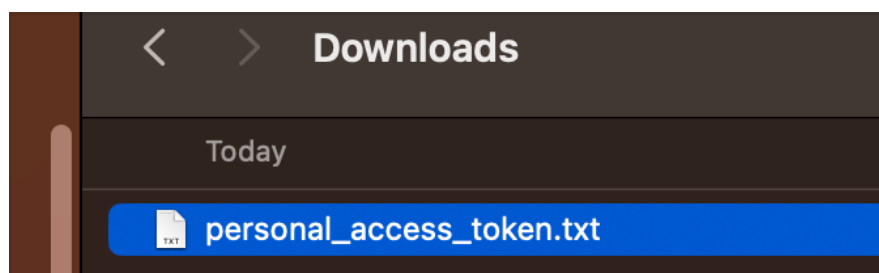
python

Expiry

In 6 months

Access	No Access	Read Only	Read/Write
Select All	☐	☐	☒
Account	☐	☐	☒
Databases	☐	☐	☒
Domains	☐	☐	☒
Events	☐	☐	☒
Firewalls	☐	☐	☒
Images	☐	☐	☒
IPs	☐	☐	☒
Linodes	☐	☐	☒
Kubernetes	☐	☐	☒
Longview	☐	☐	☒
Monitor	☐	☐	☒
NodeBalancers	☐	☐	☒
Object Storage	☐	☐	☒
StackScripts	☐	☐	☒
Volumes	☐	☐	☒

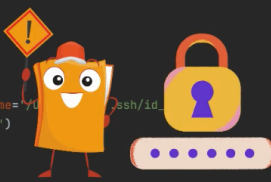
And it created a token which I downloaded



And also I can find it here: /Users/astrid/Documents/DevOps/DevOps_Core/

aws_credentials_IAM

```
monitor-website.py
26     msg = f"Application returned {response.status_code}"
27     send_notification(msg)
28
29     # restart the application
30     ssh = paramiko.SSHClient()
31     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
32     ssh.connect(hostname='172.104.252.104', username='root', key_filename='~/.ssh/id_rsa')
33     stdin, stdout, stderr = ssh.exec_command('docker start ee6b82b80ecd')
34     print(stdout.readlines())
35     ssh.close()
36     print('Application restarted')
37 except Exception as ex:
38     print(f'Connection error happened: {ex}')
39     msg = "Application not accessible at all."
40     send_notification(msg)
41
42     # restart linode server
43     linode_api4.LinodeClient(f305e161cbcefd98468decd85e5549bb4bb387090dde01c87389bd475e2cd560)
44
45     # restart the application
46
```

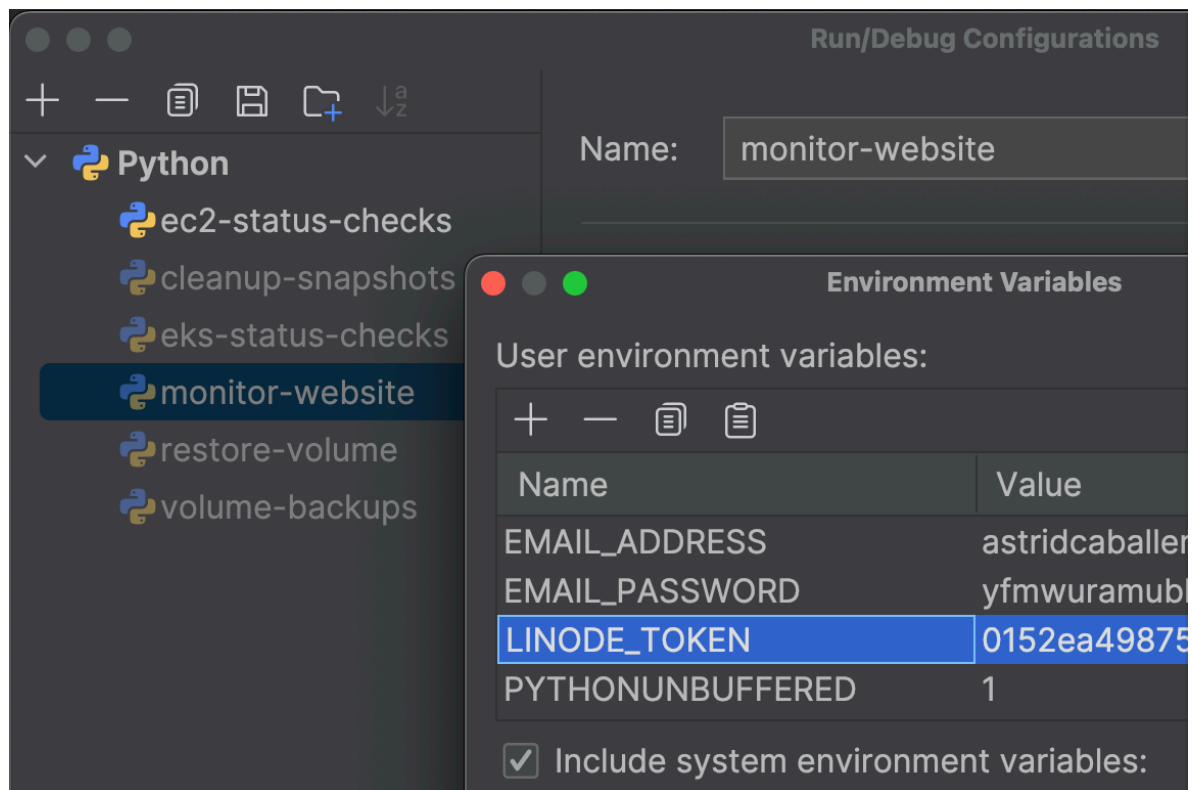


Do NOT put sensitive data inside code!

TECHWORLD WITH NANA

So we will set the token as an environment variable

```
monitor-website.py
7     EMAIL_ADDRESS = os.environ.get("EMAIL_ADDRESS")
8     EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
9     LINODE_TOKEN = os.environ.get("LINODE_TOKEN") You, 11 minutes ago • Uncommitted changes
10
11 > def send_notification(email_msg):...
12
13
14
15
16
17
18
19
20     try:
21         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
22         if response.status_code == 200:...
23
24         else:...
25
26
27
28
29     except Exception as ex:
30         print(f"Connection error happened: {ex}")
31         # send notification email to support
32         msg = "Application not accessible at all."
33         send_notification(msg)
34
35
36         # restart linode server
37         linode_api4.LinodeClient(LINODE_TOKEN)
38
```



So now we have a client that can authenticate with the Linode account with the provided token

```
44
45     # restart linode server
46     💡 client = linode_api4.LinodeClient(LINODE_TOKEN)
47     You, Moments ago • Uncommitted changes
```

From documentation there is .load()

pypi.org/project/linode-api4/

pip install linode-api4

The official Python SDK for Linode API v4

Navigation

- Project description
- Release history
- Download files

Project description

linode_api4

The official python library for the [Linode API v4](#) in python.

tests: repo or workflow not found | pypi package: 5.45.0 | docs: passing

Installation

```
pip install linode_api4
```

Building from Source

To build and install this package:

- Clone this repository
- `python3 -m pip install .`

Verified details

These details have been [verified by PyPI](#)

Project links

- Homepage
- Repository

GitHub Statistics

- Repository
- Stars: 142
- Forks: 87

github.com/linode/linode_api4-python

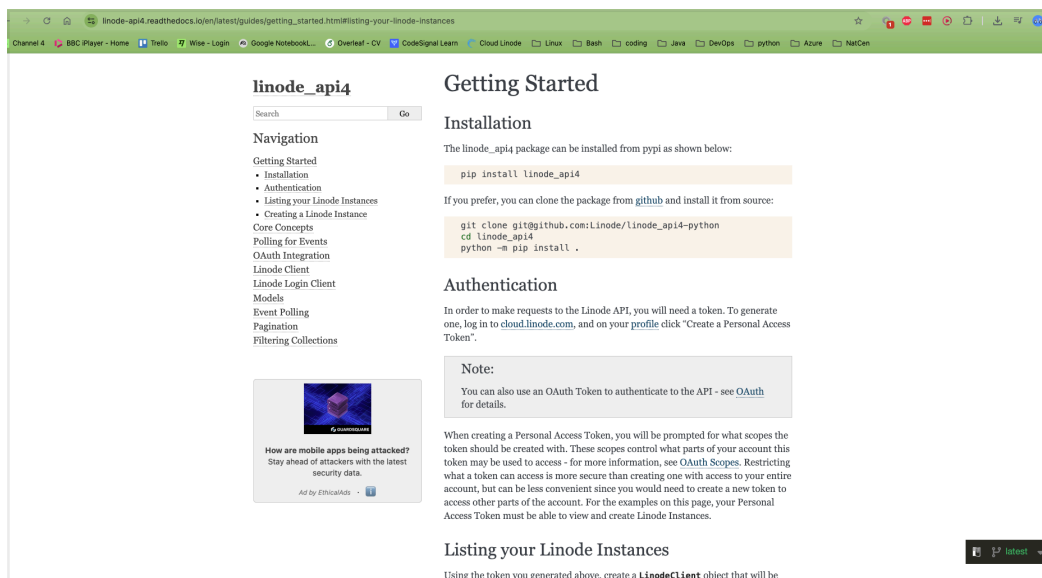
BBC iPlayer - Home | Trello | Wise - Login | Google NotebookL... | Overleaf - CV | CodeSignal Learn

README | Contributing | BSD-3-Clause license

```
# Delete the new instance
new_linode.delete()
```

Check out the [Getting Started guide](#) for more details on getting extensive documentation.

Examples



Load(*target_type*, *target_id*, *target_parent_id*=None) [source]

Constructs and immediately loads the object, circumventing the lazy-loading scheme by immediately making an API request. Does not load related objects.

For example, if you wanted to load an **Instance** object with ID 123, you could do this:

```
loaded_linode = client.load(Instance, 123)
```

Similarly, if you instead wanted to load a **NodeBalancerConfig**, you could do so like this:

```
loaded_nodebalancer_config = client.load(NodeBalancerConfig, 4
```

Parameters:

- **target_type** (*type*) – The type of object to create.
- **target_id** (*int* or *str*) – The ID of the object to create.
- **target_parent_id** (*int*, *str*, or *None*) – The parent ID of the object to create, if applicable.

Returns: The resulting object, fully loaded.

The load function can be used to connect us to a resource on Linode. In our case a server. And we can tell it with type of server we want to connect to.

So we want to connect to a Linode instance

```

45         # restart linode server
46         client = linode_api4.LinodeClient(LINODE_TOKEN)
47         client.load(linode_api4.Instance, ) You, A minute
48

```

And if we have many we want to connect to a specific one using the server id
-> **Linode ID:100597079**

The screenshot shows the Linode dashboard for a specific instance. The left sidebar has a 'COMPUTE' section with 'Linodes' selected. The main panel shows the instance 'ubuntu-gb-lon' in a 'RUNNING' state. It lists specifications: 1 CPU Core, 50 GB Storage, 2 GB RAM, and 0 Volumes. It also shows public IP addresses: 172.236.15.151 and 2600:3c13::2000:8dff:fee0:1ae3. The Linode ID is highlighted as 100597079. The instance was created on 2026-07-11 08:44.

And the codes then is:

```

45         # restart linode server
46         client = linode_api4.LinodeClient(LINODE_TOKEN)
47         nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
48

```

Now we can execute a reboot on the server:

-> .reboot()

```

45         # restart linode server
46         client = linode_api4.LinodeClient(LINODE_TOKEN)
47         nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
48         nginx_server.reboot()

```

Now let's test the code. First stop the docker container

-> docker stop 52527a5428e8

```

root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
52527a5428e8   nginx    "/docker-entrypoint..." 8 hours ago   Up 2 hours   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
root@localhost:~# docker stop 52527a5428e8
Command 'docker' not found, did you mean:
  command 'docker' from deb docker.io (29.1.3-0ubuntu3~24.04.2)
  command 'docker' from deb podman-docker (4.9.3+ds1-1ubuntu0.2)
Try: apt install <deb name>
root@localhost:~# docker stop 52527a5428e8
52527a5428e8
root@localhost:~#

```

So our script will fall into the exception code and the server will be booted

```
39     except Exception as ex:
40         print(f"Connection error happened: {ex}")
41         # send notification email to support
42         msg = "Application not accessible at all."
43         send_notification(msg)
44
45         # restart linode server
46         client = linode_api4.LinodeClient(LINODE_TOKEN)
47         nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
48         nginx_server.reboot()
49
50     # restart the application | You, 3 minutes ago • Uncommitted changes
51
Run monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject
Connection error happened: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com',
Process finished with exit code 0
```

And if we check the terminal and we can see that the connection closed we didn't do anything it happened automatically because it started to reboot

```
root@localhost:~# docker stop 52527a5428e8
52527a5428e8
root@localhost:~# Connection to 172.236.15.151 closed by remote host.
Connection to 172.236.15.151 closed.
```

Later the server is running again

Linodes / **ubuntu-gb-lon** Docs

RUNNING Maintenance Policy: Migrate

Summary		Public IP Addresses	Access
1 CPU Core	50 GB Storage	172.236.15.151	SSH Access <code>ssh root@172.236.15.151</code>
2 GB RAM	0 Volumes	2600:3c13::2000:8dff:fee0:1ae3	LISH Console via SSH <code>ssh -t astridDev@lish-gb-lon</code>
Encrypted			
Public Interface Firewall: <code>python-monitoring-firewall</code> (ID: 66702895)		Interfaces: Linode	
Plan: Linode 2 GB	Region: GB, London 2	Linode ID: 100597079	Created: 2026-07-11 08:44

Add A Tag +

And we can access it again via ssh

```
> ssh root@172.236.15.151
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 11 05:12:15 PM UTC 2026

System load:          0.61
Usage of /:           6.7% of 48.63GB
Memory usage:        9%
Swap usage:          0%
```

But the application is not running yet

```
root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS       NAMES
```

And we can reuse the code we have to restart container, so to avoid code duplication we refactor and create a new function called `restart_container()`

```
monitor-website.py x
19
20 def restart_container(): 2+ usages new *
21     print("Restarting the application...")
22     ssh = paramiko.SSHClient()
23     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
24     ssh.connect(hostname="172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
25     stdin, stdout, stderr = ssh.exec_command("docker start 52527a5428e8")
26     print(stdout.readlines())
27     ssh.close()
28
29 try:
30     response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
31     if response.status_code == 200:
32         pass
33     else:
34         print("Application Down. Fix it!")
35         # send notification email to support
36         msg = f"Application returned {response.status_code}"
37         send_notification(msg)
38     restart_container() You, Moments ago • Uncommitted changes
39
```

And use it in line 38 🙌 and line 53 🙌

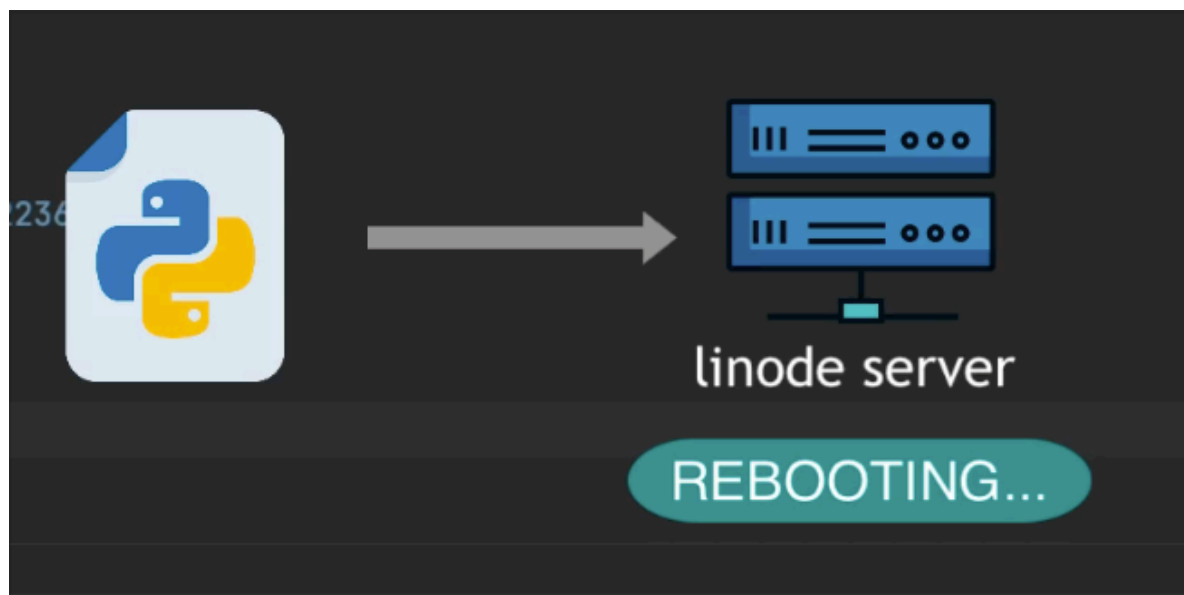
```

40     except Exception as ex:
41         print(f"Connection error happened: {ex}")
42         # send notification email to support
43         msg = "Application not accessible at all."
44         send_notification(msg)
45
46         # restart linode server
47         print("Rebooting the server")
48         client = linode_api4.LinodeClient(LINODE_TOKEN)
49         nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
50         nginx_server.reboot()
51
52         # restart the application
53         restart_container()
54

```

Now we need to take care of the timing issue -> we need to have the server up and running before we start a container. It takes time for the server to be ready.

So we will wait until server is ready:



Using a while loop

```

51
52     # restart the application
53     while True:
54         nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
55         if nginx_server.status == "running":
56             restart_container()
57             break

```

And let's test the code:

```
monitor-website.py x
47 print("Rebooting the server")
48 client = linode_api4.LinodeClient(LINODE_TOKEN)
49 nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
50 nginx_server.reboot()
51 You, A minute ago • Uncommitted changes
52 # restart the application
53 while True:
54     nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
55     if nginx_server.status == "running":
56         restart_container()
57         break
58
Run monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
Connection error happended: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', port=8080):
Sending email...
Rebooting the server
```

We check the terminal

```
root@localhost:~# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS      NAMES
root@localhost:~# Connection to 172.236.15.151 closed by remote host.
Connection to 172.236.15.151 closed.
```

We check Linode console

Linodes / ubuntu-gb-lon

REBOOTING Maintenance Policy: Migrate

Summary		Public IP Addresses	Access	
1 CPU Core	50 GB Storage	172.236.15.151	SSH Access	ssh root@172.236.15.151
2 GB RAM	0 Volumes	2600:3c13::2000:8dff:fee0:1ae3	LISH Console via SSH	ssh -t ast
Encrypted				

Public Interface Firewall: python-monitoring-firewall (ID: 66702895) Interfaces: Linode

Plan: Linode 2 GB Region: GB, London 2 Linode ID: 100597079 Created: 2026-07-11 08:44

And once is running I check the logs again:

```
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
Connection error happended: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', port=8080):
Sending email...
Rebooting the server...
Restarting the application...
['52527a5428e8\n']

Process finished with exit code 0
```

Now ssh into the server and check the app is running

```

> ssh root@172.236.15.151
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 11 05:40:35 PM UTC 2026

System load:          0.28
Usage of /:           6.7% of 48.63GB
Memory usage:        10%
Swap usage:          0%
Processes:           116
Users logged in:      0
IPv4 address for eth0: 172.236.15.151
IPv6 address for eth0: 2600:3c13::2000:8dff:fee0:1ae3

Expanded Security Maintenance for Applications is not enabled.

86 updates can be applied immediately.
76 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sat Jul 11 17:35:49 2026 from 140.228.47.252
root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS
52527a5428e8   nginx    "/docker-entrypoint..." 8 hours ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
t_elbakyan
root@localhost:~#

```

Sometimes with the while loop we still have timing issues, so let's add some extra waiting time to be 100% sure the server will be ready. As I had a timing issue with this code, even with status 'running'

```

53         # restart the application
54         while True:
55             nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
56             if nginx_server.status == "running":
57                 time.sleep(5)
58                 restart_container()
59             break
60
61

```

Run monitor-website x

```

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Autor
Connection error happended: HTTPConnectionPool(host='172-236-15-151.ip.linodeusercontent.com', port=
Sending email...
Rebooting the server...
Restarting the application...
['52527a5428e8\n']

Process finished with exit code 0

```

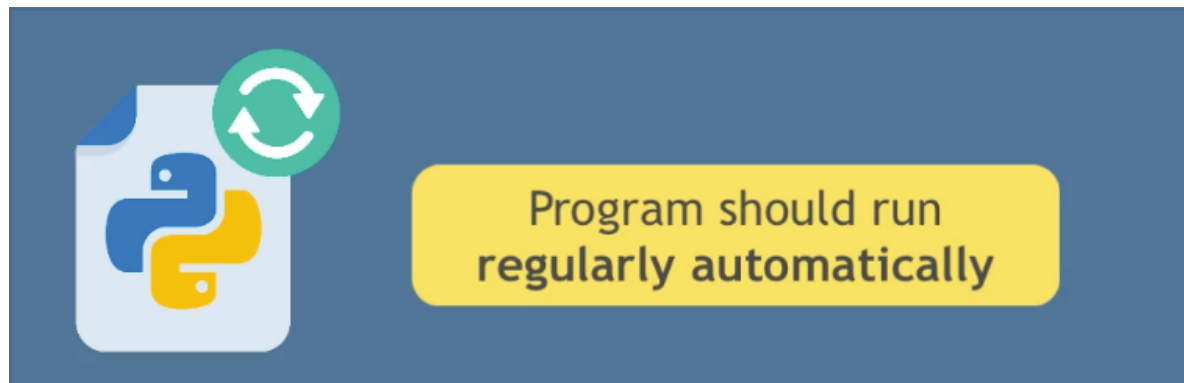
Now we can refactor the code to clean up:

```
monitor-website.py
12 def restart_server_and_container(): 1usage new *
13 # restart linode server
14 print("Rebooting the server...")
15 client = linode_api4.LinodeClient(LINODE_TOKEN)
16 nginx_server = client.load(linode_api4.Instance, target_id: 100597079)
17 nginx_server.reboot()
18
19 # restart the application
20 > while True:...
26
27 > def send_notification(email_msg):...
35
36 > def restart_container():... You, Moments ago • Uncommitted changes
44
45 try:
46 response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
47 > if response.status_code == 200:...
49 > else:...
55
56 except Exception as ex:
57 print(f"Connection error happened: {ex}")
58 # send notification email to support
59 msg = "Application not accessible at all."
60 send_notification(msg)
```

And the try/except block is cleaner.

Schedule Monitoring task

To monitor the application the python script needs to be running regularly



So now the try/except block will be in s function that the scheduler can call:

```
Automation_with_python Module_14/Automate_Python_chapter-15 monitor-website
monitor-website.py
9 EMAIL_PASSWORD = os.environ.get("EMAIL_PASSWORD")
10 LINODE_TOKEN = os.environ.get("LINODE_TOKEN")
11
12 > def restart_server_and_container():...
26
27 > def send_notification(email_msg):...
35
36 > def restart_container():...
44
45 def monitor_application(): new *
46     try:
47         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
48     >     if response.status_code == 200:...
50     >     else:...
56
57     except Exception as ex:
58         print(f"Connection error happened: {ex}")
59         # send notification email to support
60         msg = "Application not accessible at all."
61         send_notification(msg)
62         restart_server_and_container()
63 You, 2 minutes ago • Uncommitted changes
64
```

And now we create the scheduler and call the monitor_application() function:

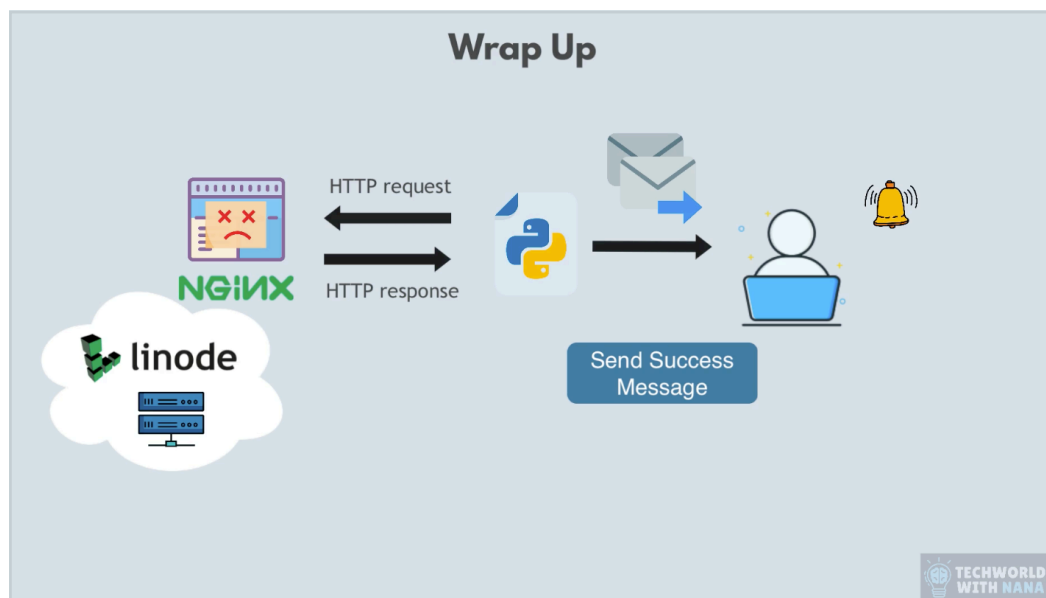
```
Automation_with_python Module_14/Automate_Python_chapter-15 monitor-website
monitor-website.py
37 > def restart_container():...
45
46 def monitor_application(): 1 usage new *
47     try:
48         response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
49     >     if response.status_code == 200:...
51     >     else:...
57
58     except Exception as ex:
59         print(f"Connection error happened: {ex}")
60         # send notification email to support
61         msg = "Application not accessible at all."
62         send_notification(msg)
63         restart_server_and_container()
64
65     schedule.every(5).minutes.do(monitor_application)
66
67     while True:
68     >     schedule.run_pending() You, Moments ago • Uncommitted changes
69
70
```

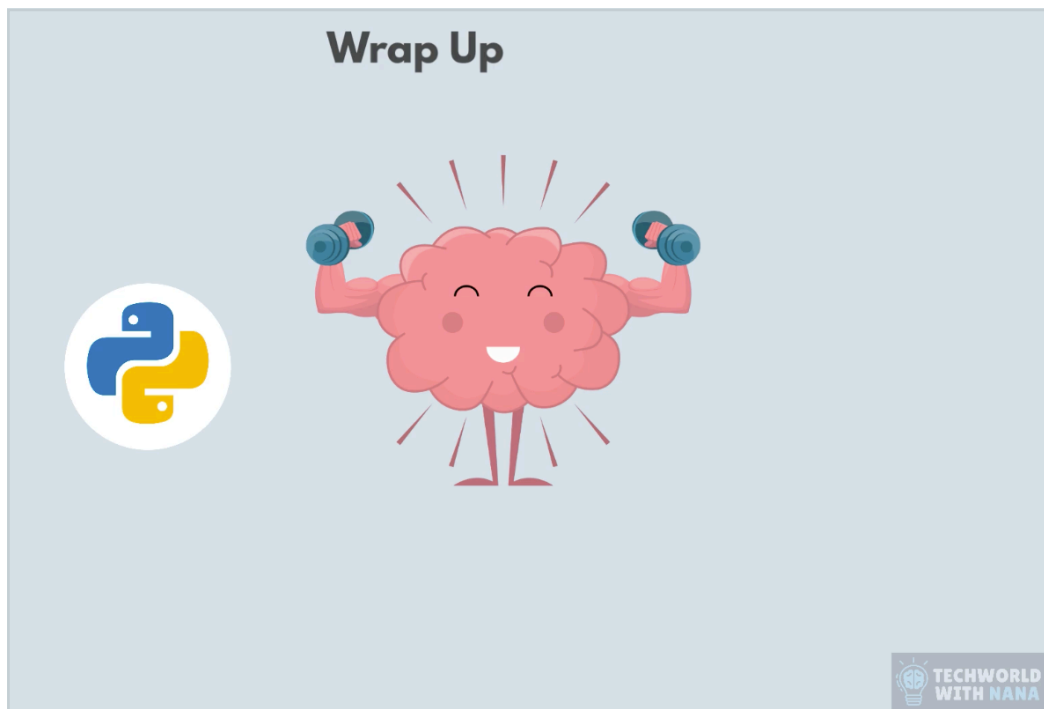
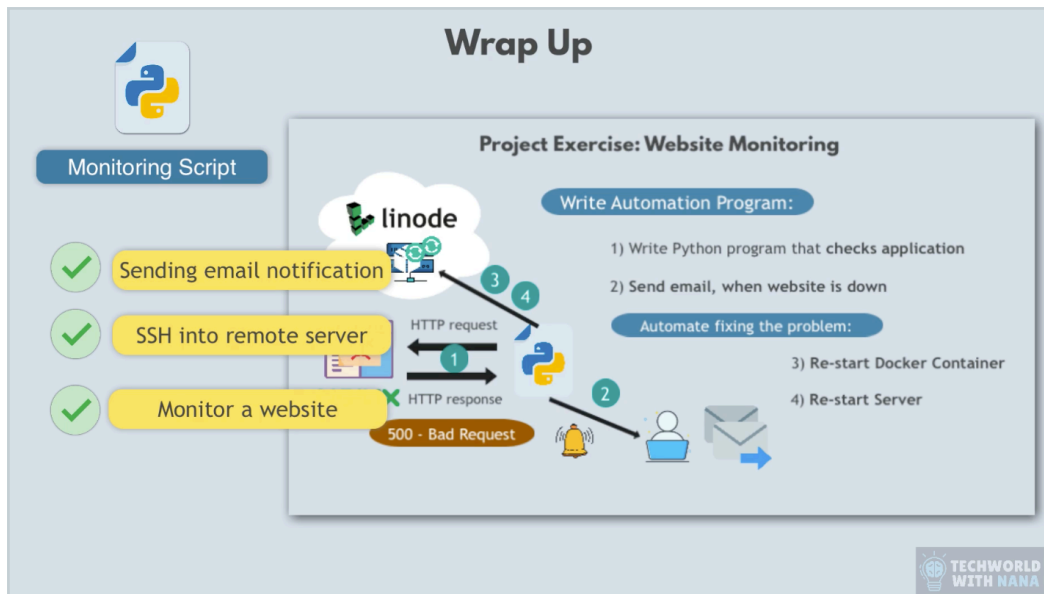
And we can test it using seconds:

```
Automation_with_python  Module_14/Automate_Python_chapter-15  monitor-webs...
monitor-website.py x
64
65     schedule.every(5).seconds.do(monitor_application)
66
67     while True:
68         ⚡ schedule.run_pending()  You, Moments ago • Uncommitted changes
69
70
71

Run  monitor-website x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /User
Application is running successfully!
```

WRAP UP





Questions:

Q:

I created a linode server and when I ssh it asks me for password every single time. now I am writing code in python and I have this ssh = paramiko.SSHClient()

```
ssh.connect("172.236.15.151", username="root", key_filename="/Users/astrid/.ssh/id_ed25519")
```

but I wonder if my server will demand the password as well

A:

Good instinct to check — and the fact that your regular ssh already asks for a password every time is the tell: your public key isn't yet authorized on the server, so key-based auth will fail for paramiko too, just in a different way.

What paramiko will actually do, unlike your terminal: it won't pause and prompt you for a password. If the key in `key_filename` isn't accepted by the server, and you haven't passed a `password=` argument, `connect()` will just raise an exception:

```
paramiko.ssh_exception.AuthenticationException: Authentication failed.
```

It has no interactive fallback — it either authenticates with what you gave it, or it errors out.

The real fix is server-side: your public key (`id_ed25519.pub`) needs to be added to `~/.ssh/authorized_keys` on the Linode box for root. If it's not there yet, that's why manual SSH keeps asking for your password — key auth is being attempted and silently failing, then SSH falls back to password.

To fix it, from your Mac:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@172.236.15.151
```

(It'll ask for the password one last time to install the key.) Then test:

```
ssh -i ~/.ssh/id_ed25519 root@172.236.15.151
```

If that logs you in with no password prompt, your paramiko code will work exactly the same way — same key, same mechanism.

Also worth checking on the server if `ssh-copy-id` isn't available or doesn't work: permissions matter a lot for SSH key auth — `~/.ssh` should be 700 and `~/.ssh/authorized_keys` should be 600 on the server, or SSH will refuse to use it even if the key is present.

Q:

```
root@localhost:~# cat ~/.ssh/authorized_keys
root@localhost:~#
```

A:

Confirmed — that file is empty, so no key has ever been authorized. That's exactly why every login (both manual ssh and your paramiko script) falls back to password.

You're currently sitting in a shell on the server itself (`root@localhost`), so the simplest fix is right here — paste your public key into that file:

1. Get your public key — open a separate terminal on your Mac (not this SSH session) and run:

```
cat ~/.ssh/id_ed25519.pub
```

It'll print one line starting with `ssh-ed25519 AAAA...` and ending in something like `astrid@....`

2. Paste it into `authorized_keys` on the server — back in this root SSH session:
`echo "ssh-ed25519 AAAA...your-full-key-here... astrid@yourmac" >> ~/.ssh/authorized_keys`
(Use the exact string from step 1 — don't retype it.)

3. Fix permissions — SSH refuses to use keys if permissions are too open:
`chmod 700 ~/.ssh`
`chmod 600 ~/.ssh/authorized_keys`

4. Test from your Mac, in a fresh terminal:
`ssh -i ~/.ssh/id_ed25519 root@172.236.15.151`
If that logs in with no password prompt, your paramiko script will too.

Alternative to steps 1–3: from your Mac (not this session), `ssh-copy-id -i ~/.ssh/id_ed25519.pub root@172.236.15.151` does all of that for you in one command — it'll ask for the password one last time, then install the key automatically.

Q:

I wonder if I miss something when I created the ssh key at the moment of the linode server creation as I can see the key created in Linode and it has the public key value stored there

A:

Yes, that's very likely the missing piece. Linode's "SSH Keys" feature (under Account/Profile settings) works like this:

- It's a key storage/convenience feature, not a live sync to your servers. When you add a public key there, Linode just remembers it for future use.
- That stored key only gets injected into `authorized_keys` at the moment you create (or rebuild/redeploy) a Linode, if you explicitly select it in the "SSH Keys" checkbox during that creation flow.
- If you added the key to your Linode account after the server already existed, or didn't tick the checkbox for that specific key during the creation wizard, it never got copied onto the actual running server — which matches exactly what you found: the key exists in Linode's UI, but `authorized_keys` on the server itself is empty.

So nothing's broken — Linode storing the key just means "this key is available to select next time you deploy something," not "this key is now on all my servers."

Two ways forward:

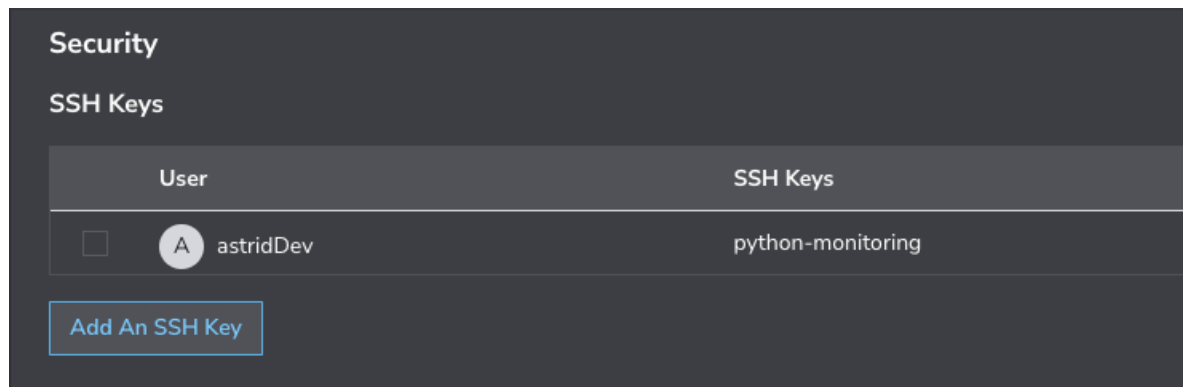
1. What we just did (manually append it to `~/.ssh/authorized_keys` on the running server) — this works regardless of the Linode UI and is the standard fix for an already-running server.

2. If you ever spin up a new Linode in the future, you can select that stored key during the creation wizard and it'll be pre-installed automatically — no manual step needed.

Since your server already exists, stick with the manual fix from the previous step — that's the correct path here.

Q:

so here is what I missed I didn't select it:



A:

Yep — that confirms it exactly. The checkbox next to astridDev with key python-monitoring is unticked, which is what determines whether that key gets written into `authorized_keys` at creation/rebuild time. Since it was unchecked when you created this Linode, it never got installed on the server — matching the empty file you found.

For this existing server, that screen won't retroactively help (it's a creation/rebuild-time setting) — the manual `authorized_keys` fix from a few messages back is still the way to go. But now you know: next time you spin up a Linode, tick that box for python-monitoring and it'll be pre-installed, no manual step needed.

Q:

for this `echo "ssh-ed25519 AAAA...paste-your-full-key-here... astridDev" >> ~/.ssh/authorized_keys` i should have use python-monitoring instead of astridDev, can I fix it? also for the linode ssh key python-monitoring can I delete it?

A:

Two separate things here — let's take them one at a time.

1. Fixing the comment in `authorized_keys`

First, check what's actually in there right now, since I want to be sure before suggesting an edit:

```
cat ~/.ssh/authorized_keys
```

The comment (astridDev, astridcaballero@gmail.com, python-monitoring, whatever) is purely cosmetic — SSH ignores it entirely for authentication, it's just a label for humans to identify keys by. So this isn't a functional bug, just a naming cleanup if you want it.

If cat confirms that line is the only line in the file, the simplest fix is to overwrite the whole file with the corrected line (note: single `>`, not `>>`, since you want to replace it, not append another copy):

```
echo "ssh-ed25519 AAAA...paste-your-full-key-here... python-monitoring" >
~/.ssh/authorized_keys
```

Only do this if cat showed just that one line — if there were other keys in there, you'd want to edit just that one line instead (e.g. with nano `~/.ssh/authorized_keys`) rather than overwrite the whole file.

2. Deleting python-monitoring from Linode's account SSH Keys page

Yes, safe to delete — it's independent of what's actually on the server. That page only stores a copy of your public key for Linode's own convenience (to offer as a checkbox during future server creation/rebuild, as you saw in the screenshot). Deleting it there:

- Does not touch `~/.ssh/authorized_keys` on your existing server — that's already set and works independently now.
- Does not touch your local key files (`~/.ssh/id_ed25519 / .pub`) on your Mac.
- Only means that stored key won't be offered as a one-click option next time you spin up a new Linode.

So it's a harmless cleanup either way — keep it if you might create more Linodes and want the convenience, delete it if you'd rather tidy up the account page.

Q:

if i don't edit the `authorized_keys` wil, I see astridDev ssh key in the linode ssh keys list?

A:

No — those two things are completely disconnected, so editing (or not editing) `authorized_keys` on the server has zero effect on Linode's account SSH Keys page.

To be precise about what each one is:

- `~/.ssh/authorized_keys` on the server — a plain text file that lives inside your Linode server's filesystem. SSH reads it locally, on that machine, to decide which keys are allowed to log in. Nothing about this file is reported back to Linode's UI.
- Linode's account "SSH Keys" page (the screenshot you showed earlier) — a

list Linode stores separately in your Linode account settings, populated only by keys you manually add there via the "Add An SSH Key" button. It has no visibility into what's actually inside any server's `authorized_keys` file.

So whatever comment/label you leave on the line in `authorized_keys` (astridDev, python-monitoring, whatever) will never show up on that Linode account page — and conversely, deleting or renaming an entry on that account page will never change anything inside your server. They're two independent stores that happen to serve a related purpose (getting your key onto a server) but don't sync with each other after the fact.

Q:

`exec_command()` returns three file-like objects (paramiko wraps the SSH channel's streams so they behave like Python's built-in `open()` result) — mirroring how any process has three standard streams: input, normal output, and error output.

```
stdin, stdout, stderr = ssh.exec_command("docker ps")
```

- `stdin` — a write-only stream connected to the remote process's standard input. You'd use this to send data into the command if it expects interactive input (e.g., typing "yes" to confirm a prompt). `docker ps` doesn't read from `stdin`, so you'd never touch this here — but the mechanism looks like:

```
stdin.write("some input\n")
stdin.flush()
```

- `stdout` — a read-only stream carrying whatever the remote command printed as normal output.

```
output = stdout.read().decode()
print(output)
```

- For `docker ps`, that might print something like:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS	NAMES			
a1b2c3d4e5f6	nginx	"nginx -g 'daemon of..."	2 hours ago	Up 2 hours
0.0.0.0:80->80/tcp	web			

- `stderr` — a read-only stream carrying whatever the remote command printed as error output — separate from `stdout` so you can distinguish "the command ran and this is its result" from "something went wrong."

```
errors = stderr.read().decode()
print(errors)
```

- If, say, Docker weren't installed on that server, `stderr` would contain something like:

```
bash: docker: command not found
```

- while `stdout` would be empty.

A couple of practical notes:

- `.read()` returns bytes, not a string — hence `.decode()` to get readable text. You

can also use `.readlines()` to get a list of individual lines instead of one big blob.

- To check whether the command actually succeeded (not just whether it printed to `stderr`), use the exit code:

```
exit_status = stdout.channel.recv_exit_status() # blocks until the command finishes; 0 = success
```

Full example:

```
stdin, stdout, stderr = ssh.exec_command("docker ps")
```

```
output = stdout.read().decode()
```

```
errors = stderr.read().decode()
```

```
exit_status = stdout.channel.recv_exit_status()
```

```
print("OUTPUT:", output)
```

```
print("ERRORS:", errors)
```

```
print("EXIT CODE:", exit_status)
```